

Ballast - My Game Design Report

Yu Han · CSCI 5999B · Project 1

Godot 4.7.2 · 2D · Underwater

1. Project overview

Item	Summary
Concept	Guide three baby fish home in each underwater dive
Genre	Single-player 2D exploration, resource management and escort
Theme	Underwater rescue
Engine / platforms	Godot 4.7.2 / Windows, macOS, Linux
Levels	Three: Reef, Kelp Drift, Trench
Target length	Roughly 5-10 minutes
Repository	https://github.com/yhan3-ctrl/ballast
Release	https://github.com/yhan3-ctrl/ballast/releases/tag/v1.0.0

2. Concept, genre and background

I made Ballast as a short underwater rescue game. The player controls Yuun the Jadenfin, finds three baby fish in each level, and brings them to a golden house. Touching a baby rescues it and makes it follow. The challenge is getting the whole group home while managing air, using currents and avoiding hazards.

I would describe the genre as a single-player 2D exploration and resource-management game with escort challenges. I chose rescue as the goal because my earlier version felt too much like swimming from one end of a map to the other. Having small fish follow the player gives the trip a purpose and makes danger more personal.

Yuun is connected to my name, Yu Han. The name begins with "Yu" and ends with the "n" from Han, while the doubled "u" echoes my handle, hnuu. The jade colour refers to jade in my Chinese name, Han Yu, and also plays on the similar sound of fish, meaning fish. I included a small H-shaped marking on Yuun.

The world and the goal

The background is simple: a surge scattered young fish across a reef, kelp channels and a deep trench. Yuun carries a lantern and guides them back to shelter. Each level has its own group of three babies, so completing all three levels means bringing nine babies home.

The golden houses are shelters, anchors are recovery points, and vents refill air. Pearls are optional treasures along the way. Glimmer, the jellyfish-like creature, reacts to nearby fish and light. I treat it as a territorial sea creature rather than an enemy that needs to be killed. The player has to move around it or draw it away.

I put the rescue premise on the map and first introduction. I kept the rest of the story light so the player could understand the goal and start playing quickly. The air system is a game rule for planning movement; it is not meant to teach real fish biology or realistic buoyancy.

3. What I want the player to learn

I want players to learn how to choose a route under a limited resource budget. Swimming harder is not always better: releasing the movement keys in a current saves air, and a longer route can be safer than a direct one. Pearls offer an extra reward, but collecting every pearl is not necessary to finish.

The other skill is reading danger before reacting. Glimmer prepares before charging, and anemones warn before releasing a pulse. The player needs to notice these signals, move at the right time and consider the babies following behind. Light can attract Glimmer away from the group, but it does not make anyone invincible.

4. How to play

Choose Dive 1 on the map and press Start Challenge. Rescue three babies by touching them, then reach the golden house. The next level unlocks after completion. Cleared levels can be replayed.

Input	Action
W / S	Upward / downward thrust
A / D	Horizontal thrust
Space	Lantern toggle
R	Retry from the active anchor; unbanked rewards are lost
Escape	Pause / resume; return from an introduction to the map
Q while paused	Return to the map
M / N	Toggle music / effects
F1	Developer diagnostics
Mouse	Choose chapters and introduction/result buttons

I keep the timer off the main play screen because this is not a race. Time appears in the results, and a best time appears after completing a level. It does not affect the score. Pausing stops the timer; retrying an anchor does not erase the time already spent. Starting the whole level again resets that attempt's time, score and rescue progress.

5. Mechanics and rules

Air and movement

I use air as both fuel and health. I want players to think about their route instead of holding a direction until they reach the exit.

A full tank lasts about 37 seconds of continuous horizontal thrust with the lantern off, provided there are no hits, refills or opposing currents. This is a reference case, not a fixed countdown for every route.

Air begins at 100, never exceeds 100, and reaching zero begins a 1.5-second death sequence before respawn. Baseline drain is 0.5 air/second. Horizontal thrust adds 2.2, upward thrust 4, downward thrust 3, and a lit lantern 1.3. W and S together cancel vertical thrust but both costs are charged. Opposing PUSH/RIP currents doubles thrust cost, not baseline or lantern cost. Diagonal inputs apply both components and their respective costs. Drag is 3.5, thrust 470 with a small sustained-input boost; speed is capped at 235 px/second.

DRIFT carries the player, PUSH resists opposing movement, and RIP enforces at least 45 px/second along its direction while within its region. Currents remain visible without light. Walls are solid. Releasing movement saves thrust expenditure; it does not stop baseline air drain.

Vents, anchors and retry

Each vent adds 40 air once. Full-air contact does not consume it; overflow is discarded. A used vent resets when the player retries its segment. Re-entering a vent while air is below maximum is required after a full-air contact; remaining stationary on it is not a refill loop.

Only a previously inactive forward anchor activates. Activation banks pending rewards, awards the remaining-air bonus, saves a spawn point and refills air. Re-crossing an activated anchor cannot refill air or score again. Each level exit settles rewards once using the same formula. Death or R restores full air, position and current-segment vents/creatures; pending pearls return to the room. Already banked pearls/points persist within the current chapter attempt. Retry count includes manual R. Dying players cannot collect vents/pearls or activate anchors/exits.

Light, Glimmer and coral

The lantern illuminates nearby geometry and attracts hostile Glimmer to Yuun. It grants no protection to Yuun or babies. Rock outlines, current arrows and hostile warning glows remain visible in darkness. Use light to bait a charge away from followers, then move out of its facing direction when it flashes.

Hostile Glimmer patrols/returns at 85 px/second. It prioritizes a lit player within 220 px; otherwise a rescued baby within 160 px or a nearby player within 100 px can trigger an attack, with clear wall line of sight required. It locks a wall-clipped line up to 260 px long, stops for 1.2 seconds of flashing and tucked-tentacle warning, charges at 330 px/second without retargeting, and rests harmlessly for 1.3 seconds, then must return home before targeting again. Only charging contact deals damage: 30 player air with knockback/grace, or one baby health. Walls stop the charge. Harmless observation Glimmer retains the light-attraction/return demonstration.

The Reef includes the same hostile charge rule used in later chapters, preceded by a short landmark hint. Its open encounter gives room to practice luring and dodging before the island routes and trench gates. The separate optional test room retains a harmless light-response specimen; it is not the campaign teaching substitute.

Stinging coral removes 25 air, pushes the player away, and grants 0.9 seconds of damage immunity. A lethal hit records the coral cause. Respawn grants 0.7 seconds of grace. Low air adds a vignette and heartbeat, never a movement penalty. Death text identifies the cause.

Pulses and follower recovery

Touching a waiting baby rescues it and restores up to 25 air once. The rescue persists after an anchor retry, so this reward cannot be collected repeatedly by retrying. Each chapter has three babies. All three must be rescued before home contact completes the chapter.

Anemones repeat a five-second cycle: 2.5 seconds safe (PASS NOW), one second warning (WAIT...), then 1.5 seconds active (PULSE!). Active pulses affect Yuun within 78 px for 20 air and knockback, with a local 1.5-second hit cooldown and normal player hit immunity. A follower within 72 px loses one health if outside its two-second hit grace. Wall line of sight is required. Pause/death stop pulse updates; retry resets pulses to their safe phase. Pulse zones are separated from essential vent/home centers.

Followers follow sampled positions from Yuun's actual trail, with shorter spacing than the earlier escort prototype. Only rescued followers take damage; unrescued babies are not killed while waiting offscreen. Coral contact, charging Glimmer and pulses each remove one of two health points, never more than once per two-second follower grace period. A zero-health follower requests a family retry, not permanent loss. Retry restores both health points and two seconds of grace to rescued followers at the anchor. Reaching a new anchor does not itself heal follower health. Lamp use never blocks damage.

Pearls and score

Pearls are optional. Each pickup gives pending points: 100 multiplied by a chain capped at x5. Another pickup within 3.5 seconds advances the chain; timeout, banking or retry resets it. Unbanked pearls/points are displayed separately from permanent rewards. The first pickup explicitly explains banking at the next anchor. Death/R removes those pending points and restores those pearls; it does not permit farming pearl points followed by a full-air retry bonus.

New anchors and the exit add $\text{round}(\text{remaining_air}) \times 20$ points before refill. I use this bonus to reward saving air as well as collecting pearls. Best chain can include a chain from a failed attempt; it is descriptive and awards no extra points.

Three levels of rules

Following Salen and Zimmerman (chapter 12), my operational rules are the controls and visible consequences described above. My constitutive rules are the underlying state changes: air is bounded from 0 to 100, each baby has two health points, a chapter requires all three rescues and home contact, and pending rewards settle only once at a new anchor or exit. My implicit expectations are that players use the normal controls without modifying the game or its save data. I do not rely on an unwritten rule to prevent score farming; the banking and reset code enforces that boundary.

6. How the three levels build on each other

I use the Reef to introduce the rules, the Kelp Drift to offer route choices, and the Trench to combine the same skills in tighter spaces. I do not introduce a new control only at the end of the game.

Level	What the player does	How it develops the challenge
The Reef	Rescue babies, refill air, use currents, avoid coral and a timed pulse, encounter a charging Glimmer	Introduces the main rules with more room to move
The Kelp Drift	Choose between upper routes and lower currents while escorting babies	Reuses movement, timing and light around branching routes
The Trench	Change depth through alternating passages and use the final upward current	Combines air planning, dodging and escorting in constrained spaces

The same rescue and banking rules apply throughout. Currents and pulse hazards appear in the first level before being reused later. The Reef also includes a real hostile Glimmer encounter, so the second level is not the first time a player sees its charge. Short hints support these encounters instead of stopping play for a long tutorial.

I gave each level its own colour palette and music arrangement. Together with the map and result screens, these make the levels feel like separate dives rather than one continuous corridor.

7. Design schemas

Information

I use Salen and Zimmerman's *Games as Systems of Information* (chapter 17) to distinguish information players can inspect from information they have not yet discovered. Ballast exposes local air and warning states clearly, while the unexplored route remains incomplete knowledge. Clear information can still create meaningful choices; hiding more information is not automatically better.

I understand this schema as looking at what information the game gives the player and how it changes their decisions. In Ballast, current arrows show flow direction, the air meter shows the remaining resource, and the babies' dots show their health. A flashing Glimmer or changing anemone gives the player information about what will happen next.

I keep essential wall outlines and danger cues readable in darkness. The lantern improves the nearby view but also attracts Glimmer, so choosing to reveal more of the area has a cost. The player uses that information to decide where and when to move.

Uncertainty

Salen and Zimmerman's *Games as Systems of Uncertainty* (chapter 15) gives me a way to discuss uncertain outcomes without adding random attacks. I apply it to incomplete route knowledge and the difficulty of executing a safe escort.

This schema looks at what the player does not yet know or cannot predict with complete confidence. My enemies follow fixed rules rather than choosing random attacks. The uncertainty comes from learning the route and judging whether a movement or detour will work.

For example, the player may know that Glimmer charges straight but still be unsure whether the whole group can get out of the way in time. Repeated attempts make that situation more understandable. I want failure to give useful information rather than feel arbitrary.

Cybernetic systems

I apply the sensor, comparator and activator model from Salen and Zimmerman's *Games as Cybernetic Systems* (chapter 18). In my player-in-the-loop model, the displayed air level is the sensor output; the player compares it with the air they expect to need; changing thrust, route or lantern use is the activating action.

A cybernetic system uses feedback: it observes a state, compares that state with a goal and acts on the difference. In my game, the air meter and low-air cues provide the information. The player compares the remaining air with the distance to safety, then changes their movement, light use or route. That action changes the air level, which produces the next round of feedback.

The player is part of this loop. A low-air warning by itself does not save anyone; it gives the player a reason to release thrust, follow a current or seek a vent. I removed low-air slowdown because it would make an already dangerous situation harder to recover from.

8. Animation and sound

I use procedural swimming, idle and death animation for Yuun, moving tentacles and warning flashes for Glimmer, and small bursts and trails for movement and pickups. The rescued babies visibly follow the player.

The game uses original synthesized music and event sounds. The three levels share a musical theme with different arrangements. Rescue, pearl pickup, refill, damage, banking and completion have distinct cues. Low air adds a heartbeat and lowers the music, so the warning is easier to notice. Players can turn music and effects off separately.

9. Playtesting and iteration

I asked three friends who regularly play games-Haopeng Chen, Roucheng Ou and Xuemeng Hu-to try Ballast. Their comments helped me see problems I missed because I already knew the routes and understood the rules. I also played through it myself.

Based on my friends' first playthroughs, I would describe the game as taking roughly **5-10 minutes** to complete. Their reported retry counts ranged from zero to six. That difference matters: knowing the controls is not the same as knowing a safe route, and I need checkpoints to make mistakes recoverable. The individual approximate times and retry counts are kept in `PLAYTEST.md`.

The most useful feedback was not just whether they liked it. At first, the goal was unclear and the game felt like swimming without a purpose. I replaced the unclear objective with recognizable baby fish and a golden home, and kept the rescue count visible. This gave the movement a reason: I was bringing someone home, not just reaching the right side of the map.

My friends also wanted clearer rocks and forks, more obstacles to weave through, and a harder third chapter. I made the wall outlines easier to see, added alternating coral, varied the trench passages and introduced timed anemone pulses. I gave the babies health so that escorting them involved a risk, rather than having them follow as decoration.

Another version used light as a shield for the babies but not for Yuun. My friends found that confusing. The jellyfish also chased continuously, which made its higher speed feel unfair. I removed the shield and changed the attack to a warning, a straight charge and a recovery period. Light now has a simpler role: it helps me see and lets me draw the creature's attention. When the visible attack line looked too artificial, I replaced it with flashing, tucked tentacles and an eye-direction cue.

The latest feedback was that there were no current problems and the game felt good overall. I used that as a reason to stop adding mechanics and focus on finishing the existing game cleanly.

How the design changed

I kept the changes in Git so I could connect each design decision to the version where I made it.

Change I made	Why I made it	Commit
Bank pearls only at a new anchor or home	Players could otherwise collect pearls, retry for full air and keep both rewards	115040b
Explain AT RISK on the first pearl pickup	Losing unbanked rewards should not feel like a bug	ebf46b7
Add a chapter map, introductions and results	The three levels needed clear beginnings and endings	c73296b
Make baby rescue the main goal	The earlier objective was hard to recognize and gave little motivation	3bd180b, a48e143
Clarify rocks, forks and dangerous creatures	Players needed to read the route while moving	a1f742a
Add follower health and timed hazards	Make the escort and final chapter more engaging	64650f5
Remove shields and replace pursuit with fixed charges	Make the rules easier to understand and attacks possible to dodge	f51bf88
Use natural warning cues and add a Reef charge encounter	Keep the warning readable and introduce the attack before later levels reuse it	f453aff

A longer version history is in docs/ITERATIONS.md.

10. Builds and verification

Alongside the friend playtests, I used automated checks to catch rule mistakes such as repeated rewards, incorrect resets, damage during pauses and attacks passing through walls. The final local candidate passed 126 rule checks. A scripted run also completed all three levels with no pearls and no retries, using normal air consumption, enemies and follower damage. Its total was about 6 minutes 43 seconds. This is a preset verification route that visits refill points and uses automated steering and dodging, not an optimized speedrun or a theoretical minimum. A human can choose a shorter or faster route. I keep this simulated time separate from the friend playtests.

I prepared Windows 32-bit and 64-bit, Linux x86_64 and macOS Universal builds for the linked v1.0.0 release. Extract the full archive before opening Ballast.exe (Windows), Ballast.x86_64 (Linux) or Ballast.app (macOS). Linux may require executable permission if the extractor does not preserve it. To run from source, import project.godot in Godot 4.7.2 and run the project. Build hashes and platform checks are listed in docs/BUILD-VERIFICATION.md; publication and submission status are in docs/SUBMISSION.md.

I used AI assistance for implementation, procedural assets, debugging and documentation, and revised the design through my own decisions and my friends' feedback. All audio is synthesized by scripts included in this repository. No commercial song recording is included.

The macOS packaged executable passed a headless startup check. I played the game locally, but this check does not establish a graphical or audio test of the downloadable archive. Windows and Linux packages passed export and archive integrity checks; they have not been launched on those platforms.

11. References and credits

Katie Salen and Eric Zimmerman, *Rules of Play: Game Design Fundamentals*, MIT Press. I reference chapter 12, "Rules on Three Levels"; chapter 15, "Games as Systems of Uncertainty"; chapter 17, "Games as Systems of Information"; and chapter 18, "Games as Cybernetic Systems". Publisher contents: <https://mitpress.ubli.sh.com/book/rules-play>

Design and authorship: Yu Han. Playtest contributors: Haopeng Chen, Roucheng Ou and Xuemeng Hu. I used AI assistance for coding, procedural visual/audio assets, debugging and document preparation; I made the design decisions and revised them using playtest feedback.